# TD 3 – – Fiche

Guillaume THÉMÈZE – Ingénieur
Enseignant contact@themeze.fr

## Plan de la séance

| | | | | | |
|---|---|---|---|---|---|
| **Durée :** | 🕒=15 min | —🕒🕒=30 min | —🕒🕒🕒=45 min | —🕒🕒🕒🕒=60 min | —🕒🕒🕒🕒🕒=75 min |
| **Difficulté :** | ⋆=très facile | — ⋆⋆=facile | — ⋆⋆⋆=moyen | — ⋆⋆⋆⋆=difficile | → ⋆⋆⋆⋆⋆=très diffic |
| **Priorité :** | ⚐=1 | — ⚐⚐=2 | — ⚐⚐⚐=3 | — ⚐⚐⚐⚐=4 | → ⚐⚐⚐⚐⚐=5 |
| **Statut :** | ○ facultatif | — | — | — | — ⊙ essentiel |

| **nº** | **Exercice** | **Type** | **Diff.** | **Priorité** | **Durée** | **Statut** |
|---|---|---|---|---|---|---|
| 1 | Transformée de Fourier discrète (FFT) d'un signal sinusoïdal | Application longue, exercice charnière du bloc | ⋆⋆⋆ | ⚐ | 🕒🕒🕒 | ⊙ **essentiel** |
| 2 | Matrices 2D : double boucle vs meshgrid | Application directe + mesure de performance | ⋆⋆ | ⚐ | 🕒🕒 | ⊙ **essentiel** |
| 3 | Trame 2D sinusoïdale orientée et FFT 2D | Application longue, exercice de synthèse du bloc | ⋆⋆⋆ | ⚐ | 🕒🕒🕒 | ⊙ **essentiel** |
| 4 | Si vous vous ennuyez... galerie de motifs avec meshgrid | Exercice facultatif, créativité et prise en main de meshgrid | ⋆⋆ | ⚐ | | ○ facultatif |

Durée totale estimée : 110 min (dont 110 min pour les exercices essentiels ; 1 exercice facultatif).

**➔ • 45 min**

## Exercice 1. Transformée de Fourier discrète (FFT) d'un signal sinusoïdal

★★★☆☆

***Application longue, exercice charnière du bloc***

◃ Plan de la séance ▲ Plan de la séance

1. Créer une fonction générant une sinusoïde (amplitude, fréquence, vecteur temps en paramètres).
2. Générer un signal sinusoïdal de fréquence 200 Hz, période d'échantillonnage 300 µs, sur un intervalle de temps de 0.1 s.
3. Afficher ce signal.
4. Calculer la TF discrète du signal (algorithme FFT).
5. Afficher le résultat (parties réelle et imaginaire). Titre, axes, légende. Préciser le rôle de `np.fft.fftshift()`.
6. Reconstruire l'axe des fréquences.
7. Afficher le module de la TF discrète. Est-ce le résultat voulu ?

FFT et ordre naturel des fréquences
Prérequis

> `np.fft.fft` calcule la Transformée de Fourier Discrète via l'algorithme FFT (rapide, $O(N \log N)$), un résultat de $N$ nombres complexes. Par défaut, l'ordre de sortie place d'abord les fréquences *positives* croissantes, puis les fréquences *négatives* croissantes : la fréquence nulle est donc en tout début de tableau, pas au centre.

Prérequis

Le spectre d'un signal réel (non complexe) vérifie $\mathrm{TF}(-f) = \overline{\mathrm{TF}(f)}$ : il est toujours symétrique par rapport à 0 Hz. Deux pics symétriques pour une sinusoïde pure sont donc normaux et attendus, pas une erreur.

Comprendre que `fftshift` doit être appliqué IDENTIQUEMENT à la TF et à l'axe des fréquences (`fftfreq`), sinon les deux courbes ne sont plus alignées indice par indice.

Retenir que la résolution fréquentielle du spectre vaut $1/$durée, indépendamment du nombre de points ou de $T_e$ seuls.

La FFT sera réutilisée telle quelle en 2D dans `td3_ex3`. Bien faire manipuler `fftshift` ET l'axe des fréquences ensemble : c'est le point le plus souvent source d'erreur silencieuse (le graphique s'affiche sans erreur Python, mais avec un axe décalé).

Génération du signal et FFT (extrait commenté)

Point de cours

```
import numpy as np
import matplotlib.pyplot as plt

def generate_sine_signal(amplitude, frequency, time_vector):
    """Genere un signal sinusoidal."""
    return amplitude * np.sin(2 * np.pi * frequency * time_vector)

frequency = 200          # Hz
Te = 300e-6              # 300 microsecondes
duration = 0.1           # s
amplitude = 3

# np.arange(0, duration, Te) : N decoule de Te et duration, cohérent
# avec l'enonce qui donne Te et la duree plutot qu'un nombre de points.
time_vector = np.arange(0, duration, Te)
N = len(time_vector)   # ~334 points (a l'arrondi flottant pres)

signal = generate_sine_signal(amplitude, frequency, time_vector)

# np.fft.fft calcule la TFD via l'algorithme FFT : resultat COMPLEXE,
# de meme taille N que le signal d'entree.
TF = np.fft.fft(signal)
```

fftshift et reconstruction de l'axe des fréquences
Point de cours

```
# fftshift replace la frequence nulle au CENTRE du tableau, et les
# frequences dans l'ordre croissant de gauche a droite : un simple
# rearrangement d'affichage, qui ne change ni les valeurs ni leur
# interpretation physique.
TF_shifted = np.fft.fftshift(TF)

# fftfreq(N, Te) donne le vecteur des frequences correspondant a
# CHAQUE case du resultat de fft, dans le MEME ordre naturel : il
# faut donc lui appliquer EXACTEMENT le meme fftshift que TF.
freq_axis = np.fft.fftfreq(N, d=Te)
freq_axis_shifted = np.fft.fftshift(freq_axis)

plt.figure()
plt.plot(freq_axis_shifted, np.real(TF_shifted), label="Partie reelle")
plt.plot(freq_axis_shifted, np.imag(TF_shifted), label="Partie imaginaire")
plt.xlabel("Frequence (Hz)"); plt.ylabel("Amplitude de la TF")
plt.legend(); plt.grid(); plt.show()
```

Module de la TF et interprétation
Point de cours

```
module_TF = np.abs(TF_shifted)

plt.figure()
plt.plot(freq_axis_shifted, module_TF)
plt.xlim(-1000, 1000)   # zoom pour bien voir les deux pics
plt.xlabel("Frequence (Hz)"); plt.ylabel("Module de la TF")
plt.grid(); plt.show()

pic = freq_axis_shifted[np.argmax(module_TF)]
print(f"Frequence du plus grand pic detecte : {pic:.1f} Hz")
print(f"Resolution frequentielle (1/duree) : {1/duration:.1f} Hz")
```

Résultat observé : deux pics symétriques détectés à environ $\pm 200$ Hz (pic exact mesuré : $-199{,}6$ Hz, à la résolution fréquentielle près, $1/\text{durée} = 10$ Hz).

1. Écrire `generate_sine_signal(amplitude, frequency, time_vector)`, puis générer le signal demandé (200 Hz, $T_e = 300\,\mu s$, durée 0,1 s) et l'afficher.

Coup de pouce

`time_vector = np.arange(0, duration, Te)` ; le nombre de points $N$ en découle, il n'est pas fixé a priori.

Correction

`signal = amplitude * np.sin(2*pi*frequency*time_vector)` ; N $\approx$ 334 points.

Génération
Solution détaillée

Avec `duration=0.1` s et `Te=300e-6` s, on obtient $N \approx 334$ points (peut varier de 1 point selon les arrondis flottants). Le signal tracé montre bien une sinusoïde de fréquence 200 Hz sur l'intervalle de 0,1 s.

2. Calculer la TF du signal, puis afficher ses parties réelle et imaginaire après `fftshift`. À quoi sert `np.fft.fftshift()` ?

Coup de pouce

Comparez l'ordre des fréquences renvoyé par défaut par `np.fft.fft` à un ordre « naturel » croissant de $-f_{max}$ à $+f_{max}$.

Correction

fftshift replace la fréquence nulle au centre du tableau, pour un affichage plus lisible.

Rôle de fftshift
Solution détaillée

Par défaut, `np.fft.fft` renvoie d'abord les fréquences positives croissantes, puis les négatives croissantes : la fréquence nulle est donc en tout début de tableau. `np.fft.fftshift` réordonne ce tableau pour placer 0 Hz au centre et les fréquences dans l'ordre croissant de gauche à droite — un simple réarrangement d'affichage, sans effet sur les valeurs elles-mêmes.

3. Reconstruire l'axe des fréquences correspondant à `TF`, puis afficher le module de la TF. Le résultat est-il celui attendu ?

oup de pou

`np.fft.fftfreq(N, d=Te)` doit recevoir le même `fftshift` que `TF` pour rester aligné avec elle.

Correction

Deux pics symétriques à environ ±200 Hz : résultat attendu pour un signal réel.

Interprétation
Solution détaillée

On retrouve bien deux pics, situés (à la résolution fréquentielle près, ici 10 Hz) à environ $+200$ Hz et $-200$ Hz, cohérents avec la fréquence du signal généré. La présence de deux pics symétriques est normale : un signal réel a toujours un spectre symétrique par rapport à 0 Hz. Le pic n'est pas exactement à 200 Hz mais au multiple de la résolution fréquentielle le plus proche, ce qui relie directement la résolution du spectre à la durée du signal analysé ($1/\text{durée} = 10$ Hz ici), et non au nombre de points ni à $T_e$ seuls.

Appliquer `fftshift` à `TF` mais pas à `freq_axis` (ou l'inverse) : le pic semble alors apparaître à une fréquence incorrecte sur le graphique. Vérifier explicitement que les deux `fftshift` sont bien présents, l'un à côté de l'autre dans le code.

Rappeler la symétrie hermitienne du spectre d'un signal réel : $\mathrm{TF}(-f) = \overline{\mathrm{TF}(f)}$.

`np.arange(0, duration, Te)` génère parfois 333 ou 334 points au lieu d'une valeur ronde : effet d'arrondi flottant sur la borne supérieure de `arange`, sans conséquence sur le raisonnement physique.

Ce qu'il faut retenir
Bilan

Un signal réel a un spectre symétrique : deux pics, pas un seul, pour une sinusoïde pure. La résolution fréquentielle d'un spectre dépend de la **durée totale** du signal analysé ($1/\text{durée}$), pas du nombre de points seul. `fftshift` doit toujours être appliqué de façon cohérente entre le spectre et son axe des fréquences associé.

➜ • 30 min

# Exercice 2. Matrices 2D : double boucle vs meshgrid

★★☆☆☆

*Application directe + mesure de performance*

◁ Plan de la séance ▲ Plan de la séance

1. Générer une matrice 2D de taille 200 par 100 avec `np.ones()`.
2. Avec une double boucle, générer une matrice 2D (largeur 200, hauteur 100 ; $x$ de 0 à 199 pas 1 ; $y$ de 0 à 99 pas 1) dont les valeurs décrivent une sinusoïde de période 10 dans le sens vertical.
3. Visualiser avec `plt.imshow()`.
4. Mesurer le temps d'exécution de cette fonction.
5. Générer cette même matrice avec un `meshgrid`.
6. Visualiser avec `plt.imshow()`.
7. Mesurer et comparer le temps d'exécution. Conclusion ?

Une matrice de « 200 par 100 » (largeur × hauteur, comme énoncé à l'oral) a un `.shape` Numpy de $(100, 200)$ — c'est-à-dire (hauteur, largeur), l'ordre inverse de celui énoncé. Cette confusion est LE point le plus piégeux de l'exercice, à illustrer avec un dessin.

Savoir utiliser `np.meshgrid` pour générer, sans boucle explicite, deux matrices `X` et `Y` contenant respectivement les coordonnées horizontales et verticales de chaque point d'une grille 2D.

Vérifier que les deux approches donnent le même résultat (`np.allclose`) et mesurer l'écart de performance.

Ce résultat (meshgrid ≫ double boucle) prolonge directement la comparaison liste/Numpy de la séance 2 (`td2_ex3`), mais en 2D où l'écart devient encore plus flagrant. C'est la brique indispensable à `td3_ex3` (tramage) : ne pas avancer si la notion de `meshgrid` n'est pas comprise par la majorité de la salle.

Convention et double boucle (extrait commenté)

Point de cours

```
import time
import numpy as np
import matplotlib.pyplot as plt

# Convention utilisee dans tout l'exercice, coherente avec l'enonce
# ("largeur 200, hauteur 100") :
#   - largeur = nombre de COLONNES = axe horizontal = x
#   - hauteur = nombre de LIGNES   = axe vertical   = y
# Une matrice de largeur L et hauteur H a donc pour .shape (H, L).
largeur = 200   # colonnes (x : 0 a 199)
hauteur = 100   # lignes   (y : 0 a 99)
periode = 10

matrice_ones = np.ones((hauteur, largeur))   # .shape = (100, 200)

# "Sinusoide de periode 10 dans le sens VERTICAL" : la valeur ne
# depend que de y (coordonnee verticale), pas de x.
matrice_boucle = np.zeros((hauteur, largeur))

debut = time.perf_counter()
for y in range(hauteur):
    valeur = np.sin(2 * np.pi * y / periode)
    for x in range(largeur):
        matrice_boucle[y, x] = valeur
fin = time.perf_counter()
duree_boucle = fin - debut

plt.figure(); plt.imshow(matrice_boucle); plt.colorbar(); plt.show()
```

Version meshgrid (vectorisée)

Point de cours

```
debut = time.perf_counter()
X, Y = np.meshgrid(np.arange(largeur), np.arange(hauteur))
matrice_meshgrid = np.sin(2 * np.pi * Y / periode)
fin = time.perf_counter()
duree_meshgrid = fin - debut

# Verification : les deux matrices doivent etre identiques.
print(np.allclose(matrice_boucle, matrice_meshgrid))   # True

print(f"Double boucle : {duree_boucle:.6f} s")
print(f"Meshgrid      : {duree_meshgrid:.6f} s")
```

Temps mesurés (une exécution, ordre de grandeur) : double boucle $\approx 0{,}0027$ s, meshgrid $\approx 0{,}0006$ s, soit environ $\times 4$ plus rapide (le facteur croît fortement avec la taille de la matrice).

1. Générer, avec une double boucle, une matrice (hauteur 100, largeur 200) dont les valeurs décrivent une sinusoïde de période 10 dans le sens vertical, et l'afficher avec `plt.imshow()`.

Coup de pouce

La valeur ne doit dépendre que de la coordonnée verticale $y$ (numéro de ligne), pas de $x$.

Correction

matrice[y, x] = sin(2*pi*y/10), boucle sur y puis x.

Implémentation — Solution détaillée

Voir le code commenté ci-dessus. Le résultat est une image de bandes horizontales (variation uniquement verticale), avec une période de 10 pixels.

2. Générer la même matrice avec `np.meshgrid`, vérifier qu'elle est identique à la version en double boucle, puis mesurer et comparer les deux temps d'exécution.

Coup de pouce

`X, Y = np.meshgrid(np.arange(largeur), np.arange(hauteur))` donne deux matrices de forme (hauteur, largeur).

Correction

np.allclose(...) -> True ; meshgrid environ 4x plus rapide que la double boucle.

Comparaison — Solution détaillée

Les deux matrices sont identiques (`np.allclose` renvoie `True`). Le temps mesuré est d'environ $0{,}0027$ s pour la double boucle contre $0{,}0006$ s pour la version meshgrid, soit un facteur $\approx 4$ sur cette taille modeste — un écart qui s'amplifie fortement sur de plus grandes matrices, exactement comme observé en 1D à la séance 2 entre liste Python et vecteur Numpy.

Inverser `X` et `Y` (`sin(2*pi*X/10)` au lieu de `Y`) donne des bandes verticales au lieu d'horizontales, sans provoquer d'erreur Python : bug silencieux, uniquement visible sur le graphique. Comparer visuellement avec l'exemple donné dans le sujet, et vérifier `np.allclose` entre la version boucle et la version meshgrid.

Toujours vérifier `.shape` explicitement après création et le comparer à ce qui est attendu.

Bilan

`meshgrid` généralise à 2D le gain de vectorisation Numpy déjà observé en 1D à la séance 2 : à privilégier systématiquement face à une double boucle Python, l'écart de performance croissant fortement avec la taille de la matrice.

➡ • 35 min

# Exercice 3. Trame 2D sinusoïdale orientée et FFT 2D

★★★☆☆☆

*Application longue, exercice de synthèse du bloc*

◁ Plan de la séance ▲ Plan de la séance

1. Réaliser une fonction qui génère une trame 2D sinusoïdale de taille $M$ par $N$, de pas spatial `step`, selon un angle `alpha`.
2. Tester cette fonction : trame de 200 par 100, pas de 10.1, angle de 35°.
3. Utiliser `np.fft.fftshift` sur la matrice générée.
4. Faire la TF en 2D de la trame et afficher le résultat.

**Projection orientée direction quelconque** — Prérequis (étudiants)

Pour orienter une sinusoïde 2D selon un angle $\alpha$ quelconque, on projette chaque point $(x, y)$ sur la direction définie par le vecteur unitaire $(\cos\alpha, \sin\alpha)$ : la coordonnée obtenue, $x\cos\alpha + y\sin\alpha$, varie « le long des franges » et sert d'argument à un cosinus de période spatiale `step`.

**Généraliser meshgrid à un motif orienté** — Objectifs pédagogiques (étudiants)

Réutiliser la grille de coordonnées `X, Y` de l'exercice précédent pour construire un motif dont l'orientation dépend d'un paramètre.

**Interpréter une FFT 2D** — Objectifs pédagogiques (étudiants)

Relier visuellement l'orientation et le pas spatial d'une trame à la position des pics de sa transformée de Fourier 2D.

**Dernier exercice du bloc obligatoire** — Points pédagogiques (enseignants)

Il boucle la boucle entre FFT 1D (`td3_ex1`) et meshgrid (`td3_ex2`). Si le temps manque, il est possible de s'arrêter après la question 2 (génération et affichage de la trame) et de traiter les questions 3-4 (FFT2D) en fin de séance suivante plutôt que de bâcler.

**Fonction `generate_grating` (extrait commenté)** — Point de cours

```
import numpy as np
import matplotlib.pyplot as plt

def generate_grating(M: int, N: int, step: float, alpha_deg: float):
    """Genere une trame (grating) sinusoidale 2D orientee.
    M : largeur (colonnes) ; N : hauteur (lignes) ; step : pas
    spatial ; alpha_deg : angle par rapport a l'horizontale."""
    alpha = np.deg2rad(alpha_deg)

    # Meme convention que td3_ex2 : X, Y ont pour forme (N, M).
    x = np.arange(M)
    y = np.arange(N)
    X, Y = np.meshgrid(x, y)

    # Projection de chaque point sur la direction (cos a, sin a) :
    # coordonnee "le long des franges", periodisee avec un cosinus.
    projection = X * np.cos(alpha) + Y * np.sin(alpha)
    trame = np.cos(2 * np.pi * projection / step)
    return trame

M, N = 200, 100
trame = generate_grating(M, N, step=10.1, alpha_deg=35)
# trame.shape = (100, 200) = (N, M) = (hauteur, largeur)
```

fftshift spatial puis FFT 2D

Point de cours

```
# Question 3 : fftshift applique DIRECTEMENT sur l'image spatiale
# (pas encore une TF) pour observer que c'est une simple permutation
# de quadrants, independante de la signification physique du tableau.
trame_shiftee = np.fft.fftshift(trame)

# Question 4 : TF2D puis fftshift pour recentrer, affichage en
# log1p(module) pour ne pas ecraser les pics lateraux par la
# composante centrale (frequence nulle).
TF2D = np.fft.fft2(trame)
TF2D_shiftee = np.fft.fftshift(TF2D)

plt.figure()
plt.imshow(np.log1p(np.abs(TF2D_shiftee)))
plt.colorbar()
plt.title("Module de la TF2D (echelle log1p)")
plt.show()
```

Résultat attendu : deux pics symétriques par rapport au centre, alignés perpendiculairement aux franges, à une distance du centre inversement proportionnelle au pas spatial.

1. Écrire `generate_grating(M, N, step, alpha_deg)` et générer une trame de 200 par 100, pas de 10.1, angle de 35°.

   Coup de pouce

   Projetez chaque point de la grille `meshgrid` sur le vecteur directeur $(\cos\alpha, \sin\alpha)$ avant d'appliquer un cosinus.

   Correction

   projection = X*cos(alpha) + Y*sin(alpha) ; trame = cos(2*pi*projection/step).

   Implémentation — Solution détaillée

   Voir le code commenté ci-dessus. Si blocage sur le calcul de la projection, partir des cas simples $\alpha = 0°$ (franges verticales, seul $X$ intervient) et $\alpha = 90°$ (franges horizontales, seul $Y$ intervient) avant de généraliser.

2. Appliquer `np.fft.fftshift` directement sur la trame (image spatiale, pas encore une TF). Que constate-t-on ?

   Coup de pouce

   `fftshift` est une opération purement géométrique sur un tableau, indépendante de la signification physique de son contenu.

   Correction

   L'image est simplement réarrangée par permutation de quadrants (haut-gauche <-> bas-droite, etc.).

   Interprétation — Solution détaillée

   `fftshift` permute les quadrants de l'image (haut-gauche avec bas-droite, haut-droite avec bas-gauche), quel que soit le contenu du tableau : ici appliqué à une image spatiale, cela produit simplement une version « recollée » de la trame, sans aucun rapport pour l'instant avec une transformée de Fourier — bonne façon de dissocier la mécanique de `fftshift` de son usage habituel sur un spectre.

3. Calculer la FFT 2D de la trame et afficher son module (avec `fftshift` et une échelle `log1p` pour la lisibilité). Interpréter la position des pics observés.

**Coup de pouce**

La distance des pics au centre est liée au pas spatial `step`, et leur orientation à l'angle `alpha_deg` de la trame.

**Correction**

Deux pics symétriques par rapport au centre, perpendiculaires aux franges, à une distance inversement proportionnelle à step.

**Interprétation physique** **Solution détaillée**

Comme pour la FFT 1D, la trame étant réelle, sa TF2D présente une symétrie centrale : deux pics symétriques par rapport au centre de l'image, alignés perpendiculairement à la direction des franges (donc dans la direction $(\cos\alpha, \sin\alpha)$), à une distance du centre d'autant plus grande que le pas spatial `step` est petit (fréquence spatiale élevée). Si l'on double `step`, la distance des pics au centre est divisée par deux.

La même confusion qu'à l'exercice précédent : une image avec les axes inversés, sans erreur Python. Comparer avec l'exemple visuel fourni dans le sujet.

Les pics apparaissent alors dans les coins de l'image au lieu d'être centrés, rendant le résultat difficile à interpréter : toujours enchaîner `fft2` puis `fftshift` avant tout affichage de spectre 2D.

`np.fft.fft2` attend un tableau 2D : une erreur de forme (tableau 1D par erreur) ne lève pas toujours une exception explicite, donc vérifier `trame.shape` avant l'appel.

**Bilan**

Les mêmes précautions que pour la FFT 1D (`td3_ex1`) s'appliquent en 2D : appliquer `fftshift` de façon cohérente, et se rappeler qu'un signal réel (ici une image réelle) a un spectre symétrique — ici, deux pics au lieu d'un seul, alignés perpendiculairement aux franges de la trame.

**Aller plus loin**

Mettre à zéro certaines fréquences de la TF2D puis appliquer une TF2D inverse (`np.fft.ifft2`) pour visualiser l'effet sur l'image reconstruite — une première approche du filtrage fréquentiel, avec des applications réelles en moiré, réseaux de diffraction et tests de résolution optique (mires USAF).

Master – Filtrage fréquentiel simple

• 20-30 min

## Exercice 4. Si vous vous ennuyez... galerie de motifs avec meshgrid *(facultatif)*

★★☆☆☆

***Exercice facultatif, créativité et prise en main de meshgrid***

◁ Plan de la séance ▲ Plan de la séance

Si vous avez terminé en avance : écrire des fonctions qui utilisent `np.meshgrid` pour générer une image représentant :
1. un disque ;
2. des anneaux concentriques ;
3. un damier ;
4. une mire radiale rectangulaire (secteurs alternés noir/blanc) ;
5. une mire radiale sinusoïdale ;
6. une gaussienne en 2D ;
7. une spirale ;
8. une étoile. Afficher chaque image avec `plt.imshow` en niveaux de gris.

Coordonnées cartésiennes et polaires
Prérequis

Tous ces motifs s'obtiennent en calculant une formule sur la grille $(X, Y)$ fournie par `meshgrid`, sans aucune boucle. Pour les motifs circulaires, on passe en coordonnées polaires : $r = \sqrt{X^2 + Y^2}$ et $\theta = \operatorname{atan2}(Y, X)$. Un motif « tout ou rien » s'obtient par un test booléen converti en `float` ; un motif progressif par un cosinus ou une exponentielle.

Traduire une forme géométrique en une condition ou une formule appliquée à toute la grille d'un seul coup (vectorisation).

Utiliser $r$ et $\theta$ pour décrire des motifs à symétrie circulaire ou angulaire.

Facultatif : à proposer aux étudiants les plus rapides. Les motifs sont réinvestissables plus tard comme images de test pour la FFT 2D (`td3_ex3`) ou le tramage.

Grille centrée et coordonnées polaires
Point de cours

```
import numpy as np
import matplotlib.pyplot as plt

def grille(M, N):
    """Grille centree : X, Y de forme (N, M), origine au centre."""
    x = np.arange(M) - (M - 1) / 2
    y = np.arange(N) - (N - 1) / 2
    return np.meshgrid(x, y)

# Coordonnees polaires, utiles pour les motifs circulaires :
#   r     = np.sqrt(X**2 + Y**2)
#   theta = np.arctan2(Y, X)      # angle dans ]-pi, pi]
```

Disque, anneaux, damier
Point de cours

```
def disque(M, N, R):
    X, Y = grille(M, N)
    return (X**2 + Y**2 <= R**2).astype(float)

def anneaux(M, N, largeur):
    X, Y = grille(M, N)
    r = np.sqrt(X**2 + Y**2)
    # partie entiere de r/largeur : pair -> blanc, impair -> noir
    return (np.floor(r / largeur) % 2 == 0).astype(float)

def damier(M, N, c):
    X, Y = grille(M, N)
    # parite de la somme des indices de case
    return ((np.floor(X / c) + np.floor(Y / c)) % 2 == 0).astype(float)
```

Mires radiales
Point de cours

```
def mire_radiale_rect(M, N, n_secteurs):
    """n_secteurs secteurs blancs (etoile de Siemens)."""
    X, Y = grille(M, N)
    theta = np.arctan2(Y, X)
    return (np.floor(theta * n_secteurs / np.pi) % 2 == 0).astype(float)

def mire_radiale_sin(M, N, n_periodes):
    X, Y = grille(M, N)
    theta = np.arctan2(Y, X)
    return 0.5 * (1 + np.cos(n_periodes * theta))
```

Gaussienne 2D et spirale
Point de cours

```
def gaussienne(M, N, sigma):
    X, Y = grille(M, N)
    return np.exp(-(X**2 + Y**2) / (2 * sigma**2))

def spirale(M, N, n_bras, pas):
    X, Y = grille(M, N)
    r = np.sqrt(X**2 + Y**2)
    theta = np.arctan2(Y, X)
    # la phase depend a la fois de l’angle et du rayon
    return 0.5 * (1 + np.cos(n_bras * theta - 2 * np.pi * r / pas))
```

Étoile et affichage de la galerie
Point de cours

```
def etoile(M, N, R, n_branches, creux=0.3):
    X, Y = grille(M, N)
    r = np.sqrt(X**2 + Y**2)
    theta = np.arctan2(Y, X)
    # rayon limite module par l’angle : maximal (= R) au bout de chaque
    # branche, minimal (= creux*R) entre deux branches
    r_max = R * (creux + (1 - creux)
                 * (1 - np.abs(np.sin(n_branches * theta / 2))))
    return (r <= r_max).astype(float)


M, N = 300, 300
images = {
    "disque": disque(M, N, 100),
    "anneaux": anneaux(M, N, 15),
    "damier": damier(M, N, 30),
    "mire radiale rect.": mire_radiale_rect(M, N, 12),
    "mire radiale sin.": mire_radiale_sin(M, N, 8),
    "gaussienne": gaussienne(M, N, 40),
    "spirale": spirale(M, N, 3, 60),
    "etoile": etoile(M, N, 130, 5),
}
fig, axes = plt.subplots(2, 4, figsize=(12, 6))
for ax, (nom, img) in zip(axes.ravel(), images.items()):
    ax.imshow(img, cmap="gray")
    ax.set_title(nom)
    ax.axis("off")
plt.tight_layout()
plt.show()
```

1. Écrire une fonction `grille(M, N)` renvoyant une grille centrée, puis `disque`, `anneaux` et `damier`.

**Coup de pouce**

Un disque est l'ensemble des points tels que $X^2 + Y^2 \leq R^2$. Pour les anneaux et le damier, utiliser la partie entière (`np.floor`) et la parité (`% 2`).

**Correction**

Disque : X**2 + Y**2 <= R**2 ; anneaux : floor(r/largeur) pair ; damier : floor(X/c) + floor(Y/c) pair.

**Implémentation** – **ution détail**

Voir le code commenté ci-dessus.

2. Écrire `mire_radiale_rect` et `mire_radiale_sin` : secteurs alternés noir/blanc, puis variation sinusoïdale selon l'angle.

**oup de pouc**

Le motif ne dépend que de l'angle $\theta = \operatorname{atan2}(Y, X)$, pas du rayon.

**Correction**

Rect : floor(theta*n/pi) pair ; sinusoidale : 0.5*(1 + cos(n*theta)).

**Implémentation** – **ution détail**

Voir le code ci-dessus. Le motif est constant le long de chaque demi-droite issue du centre.

3. Écrire `gaussienne` et `spirale`.

**oup de pouc**

Gaussienne : $e^{-r^2/(2\sigma^2)}$. Spirale : un cosinus dont la phase dépend à la fois de $\theta$ et de $r$.

**Correction**

Gaussienne : exp(-r**2/(2*sigma**2)) ; spirale : 0.5*(1 + cos(n*theta - 2*pi*r/pas)).

**Implémentation** – **Solution détaillée**

Voir le code ci-dessus. Pour la spirale, le terme $n\theta$ donne le nombre de bras et le terme en $r$ les enroule.

4. Écrire `etoile(M, N, R, n_branches)`.

**Coup de pouce**

Définir un rayon limite qui dépend de l'angle, puis garder les points dont le rayon est inférieur à cette limite.

**Correction**

Garder les points tels que r <= R*(creux + (1-creux)*(1 - abs(sin(n*theta/2)))).

**Implémentation** – **ution détail**

Voir le code ci-dessus. Le rayon limite varie avec $\theta$ et présente $n$ maxima : un par branche.

**Oublier de centrer la grille dans un coin**

Sans recentrage (`- (M - 1) / 2`), les cercles sont centrés sur le coin haut-gauche de l'image : penser à centrer la grille.

**Oublier de convertir [illegible]**

Un test comme `X**2 + Y**2 <= R**2` renvoie un tableau de booléens ; `.astype(float)` évite des surprises lors des calculs et de l'affichage.

**Discontinuité de l'angle**

`arctan2` saute de $\pi$ à $-\pi$ sur l'axe des abscisses négatives : pour un nombre entier de périodes, le motif reste cohérent, mais pas avec un nombre non entier.

**Bilan** – **Une seule idée, plusieurs motifs**

Toutes les images de la galerie reposent sur la même recette : `meshgrid`, éventuellement conversion en $(r, \theta)$, puis une formule vectorisée. Changer la formule change la forme.

Aller plus loin

Multiplier une mire par une gaussienne (fenêtrage) ou additionner plusieurs motifs, et observer l'effet sur leur spectre.

## Bilan et prolongements du TD